

How to Use AI Prompts to Build Software

A practical client guide for working with AI coding tools
(Cursor, ChatGPT, Claude, and similar assistants)

Prepared for project collaboration
You can upload or share this PDF with your team

1. What this guide is for

AI tools can help you design, build, debug, and document software faster. They do not replace product decisions, testing, or ownership of production. This guide shows how to write prompts that get useful results, and how to avoid common traps when building a real product (API + website + billing).

Good uses

- Turning a feature idea into a clear build plan
- Writing or editing code with step-by-step instructions
- Explaining errors and suggesting fixes
- Drafting docs, emails, and release notes
- Reviewing a checklist before going live

Not enough alone

- Assuming a green UI means production works
- Skipping login / payment / real-device testing
- Shipping secrets (.env keys) into chat or git

2. The prompt formula that works

Use this structure every time:

- Context: what product, stack, and file/area you mean
- Goal: what "done" looks like
- Constraints: must / must-not (tech, design, security)
- Inputs: links, screenshots, error text, API responses
- Output: plan only, or implement, or both

Example prompt

Context: Data Captain is a US ETF API (Next.js frontend + Express API).
Goal: Add a dashboard banner that shows remaining daily API requests.
Constraints: Reuse existing usage API; mobile-friendly; no new deps.
Output: Implement the change and tell me how to verify it.

3. Prompt patterns for building

A. Discovery (before coding)

Try this

Explain how signup -> API key -> first request works in this codebase.
List the exact files and endpoints. Do not change code yet.

B. Build a feature

Try this

Implement X. Match existing patterns in the repo. Keep the diff small.
After coding, give a 5-step test plan I can run myself.

C. Fix a bug

Try this

Here is the error (paste full message + URL + screenshot).
Find the root cause first. Do not guess CORS until you check the API
health response. Then propose the smallest fix.

D. Prepare for release

Try this

Give a go-live checklist for this change: env vars, migrations,
payment/webhooks if billing, health URL, and rollback notes.

4. How to work with the AI day to day

- One job per chat thread when possible (billing, SEO, onboarding).
- Paste real errors, not summaries.
- Ask for a plan when the task is large; approve before big edits.
- After each change: run it, click through, check the network tab.
- Commit and push when a milestone works, so teammates share the same code.

5. Verification checklist (do not skip)

AI output is a draft until you prove it in production-like conditions:

- Local: page loads, no console errors
- API: GET /health or /v1/health returns 200
- Auth: signup / login / redirect still work
- Money path: test payment / plan upgrade if touched
- Mobile: check the first screen on a phone width
- Env: production variables set after deploy

6. Mistakes we see often (and how to avoid them)

- Browser says CORS -> first check if the API is actually up.
- UI says Coming soon / disabled -> check live billing config, not only frontend copy.
- Path changes (/api to /v1) -> update frontend, docs, and SDKs together.
- Multiple services (web + API) -> treat them as one product; verify both after a change.

7. Simple template you can copy

Copy / paste template

Role: You are my senior engineer for [product].
Context: [stack + what already exists]
Task: [feature or bug]
Done means: [user-visible result]
Constraints: [security, style, no big refactors]
Please: 1) short plan 2) implement 3) how I test it.

8. Next step for our project

If you want help building on Data Captain (ETF API, dashboard, billing, docs), send: (1) the goal in one sentence, (2) any screenshot or error, (3) whether this is frontend, API, or both. We will turn that into a prompt and a build plan.

This document is a practical guide for clients collaborating with AI-assisted development. It is not legal or financial advice.